



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE
COMPUTER SCIENCE DEPARTMENT

**Authenstickator: a TPM-secured TOTP authenticator app
running on a USB stick**

MASTER'S THESIS

Master graduate: **Roland BÁLINT**

Supervisor: **Conf. dr. ing. Ciprian Pavel OPRIȘA**

September, 2026



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE
COMPUTER SCIENCE DEPARTMENT

DEAN,
Prof.Dr.Eng. Vlad MUREȘAN

HEAD OF DEPARTMENT,
Prof.Dr.Eng. Rodica POTOLEA

Master graduate: **Roland BÁLINT**

**Authenstickator: a TPM-secured TOTP authenticator app running on
a USB stick**

1. **Project proposal:** *Brief description of the master's thesis and the initial data*
2. **Project contents:** *(enumeration of the main parts) Example: Presentation page, Title of chapter 1, Title of chapter 2, ..., Title of chapter n, Bibliography, Appendices.*
3. **Place of documentation:** *Example: Technical University of Cluj-Napoca, Computer Science Department*
4. **Consultants:**
5. **Date of issuing the proposal:** *Example: November, 2025*
6. **Date of delivery:** *Example: July 5th, 2026*

Master's graduate: (*here comes the signature*)

Supervisor:

**TECHNICAL UNIVERSITY**

OF CLUJ-NAPOCA, ROMANIA

**FACULTY OF AUTOMATION AND COMPUTER SCIENCE
COMPUTER SCIENCE DEPARTMENT****Declarație pe proprie răspundere privind
autenticitatea lucrării de disertație**

Subsemnatul(a) Roland BÁLINT, legitimat(ă) cu seria nr., CNP, autorul lucrării Authenstickator: a TPM-secured TOTP authenticator app running on a USB stick, elaborată în vederea susținerii examenului de finalizare a studiilor de disertație la Facultatea de Automatică și Calculatoare, Programul de studiu, din cadrul Universității Tehnice din Cluj-Napoca, sesiunea (iulie/septembrie) a anului universitar 2024-2025, declar pe proprie răspundere că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate în textul lucrării și în bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de disertație.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de disertație*.

Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine.

Data

.....

Roland BÁLINT

.....

Signature

Contents

Chapter 1	Introduction	1
Chapter 2	Objectives of the Research	2
Chapter 3	Bibliographic Research	3
3.1	Existing Solutions	3
3.2	Two-Factor Authentication	3
3.3	Encryption	3
3.3.1	LUKS	4
3.3.2	BitLocker	4
3.3.3	AES	4
3.4	TOTP	4
3.5	TPM	4
Chapter 4	Project Presentation	5
4.1	The Proposed Framework	5
4.1.1	USB Encryption	5
4.1.2	User Password	7
4.1.3	TPM	7
4.2	The Implementation	8
4.2.1	The Back-end	8
4.3	Provisioning the TPM: a Requirement for the Application	13
4.4	Development	14
4.4.1	Basic Setup	14
4.4.2	Testing with TPM on a VM	15
4.4.3	Setup TPM	15
Chapter 5	Theoretical and Experimental Results	19
5.1	Testing on Linux	19
5.2	Unit tests	19
Chapter 6	Conclusions	20
	Bibliography	21

Chapter 1

Introduction

Chapter 2

Objectives of the Research

The objective is to provide a secure, modular and portable architecture for a TOTP authenticator on a USB stick, and a software solution implementing it.

Authenstickator replaces a regular authenticator app like Microsoft's Authenticator or Google Authenticator. For these, you need a mobile phone. If you don't have a dedicated work phone, you might need to tie your personal device to the TOTP verification of a work account. In case you lose your phone, your company might require to wipe your data. Furthermore, mixing personal belongings with work is not a good practice. A USB device is cheap, so it is not an overhead to have a USB device solely for authentication purposes.

While professional solutions like YubiKeys exist, they are quite expensive due to their dedicated hardware and branding. While they are far more secure than a simple USB stick, for two-factor authentication Authenstickator is more than enough.

Chapter 3

Bibliographic Research

3.1 Existing Solutions

Virtual FIDO: runs on the host machine, no second device, kills the purpose of 2FA.

USB Raptor: turns USB into a security key; if removed, blocks the PC. Different purpose, similar goal.

YubiKey: professional, hardware-backed, but expensive.

3.2 Two-Factor Authentication

There are several types of authentication factors that can be used to confirm a person's identity.[1] The most common include:

1. **A knowledge factor:** This is information that the user knows, which could include a password, personal identification number (PIN), or passcode.
2. **A possession factor:** This is something that the user has or owns, which could be their driver's license, identification card, mobile device, or an authenticator app on their smartphone.
3. **A location factor:** This is usually guided by the location in which a user attempts to authenticate their identity. Organizations can limit authentication attempts to certain devices in specific locations, depending on how and where employees log in to their systems.
4. **A time factor:** This factor restricts authentication requests to specific times when users are allowed to log in to a service. All access attempts outside of this time will be blocked or restricted.

TOTP based 2FA builds on possession factor - "something you have". Authenticator replaces the mobile phone as possession with a USB stick, this way it does not break the idea of 2FA, unlike Virtual FIDO.

3.3 Encryption

The app uses different encryption algorithms.

3.3.1 LUKS

For encrypting the USB stick for Linux.

3.3.2 BitLocker

For encrypting the USB stick for Windows.

3.3.3 AES

For encrypting the storage file of secrets.

3.4 TOTP

The system generates tokens based on the Time-Based One-Time Password (TOTP) algorithm [2].

3.5 TPM

“A Trusted Platform Module, also known as a TPM, is a cryptographic coprocessor that is present on most commercial PCs and servers” [3].

Chapter 4

Project Presentation

4.1 The Proposed Framework

The goal of this Masters' thesis is to provide a generic framework for a TOTP application on a USB stick, as well as an implementation for it. In this section, we will focus on the framework, i.e., the ideas and principles, rather than the technology used. The presented framework could work on other devices as well, it is a generic flow of actions meant to be abstract. At the same time, the framework's main goal is security, so it has to take into account all kinds of attacks and countermeasures for them. With that in mind, the framework focuses on minimizing the possibility of exploits, errors, deadlocks.

Simply put, a TOTP application generates TOTP codes (numbers) based on TOTP secrets (base64 codes). The application which requires TOTP (example: Gmail, Facebook, etc.) displays a QR code or an equivalent secret. The user scans the QR code / introduces the secret into the TOTP application. From then on, the TOTP application stores that secret in a way or other, and keeps generating TOTP codes.

So the problem can be reduced to the following aspects/ goals:

1. **Generate the TOTP codes:** This is the main functional requirement of the framework. TOTP code generation is a standard algorithm and requires only the current time and the TOTP secret.
2. **Store the secrets safely:** Store TOTP secrets in a way which is secure and handle possible attack scenarios.
3. **Keep the 2FA aspect:** The framework treats the device on which the app runs as the possession factor of Two-Factor Authentication 3.2. Breaking this aspect would make the framework useless.

Figure 4.1 shows the two main flows of the proposed architecture. The following sections will go in depth on each aspect.

4.1.1 USB Encryption

The main security layer is provided by the device on which the app resides: the USB itself shall be encrypted. For Linux, a password-protected LUKS2-encrypted USB stick shall be used. The encryption of the device applies on its whole content. This ensures that the stored secrets cannot be read without providing the password. The passwords could be saved in plaintext, the USB encryption would encrypt it anyway.

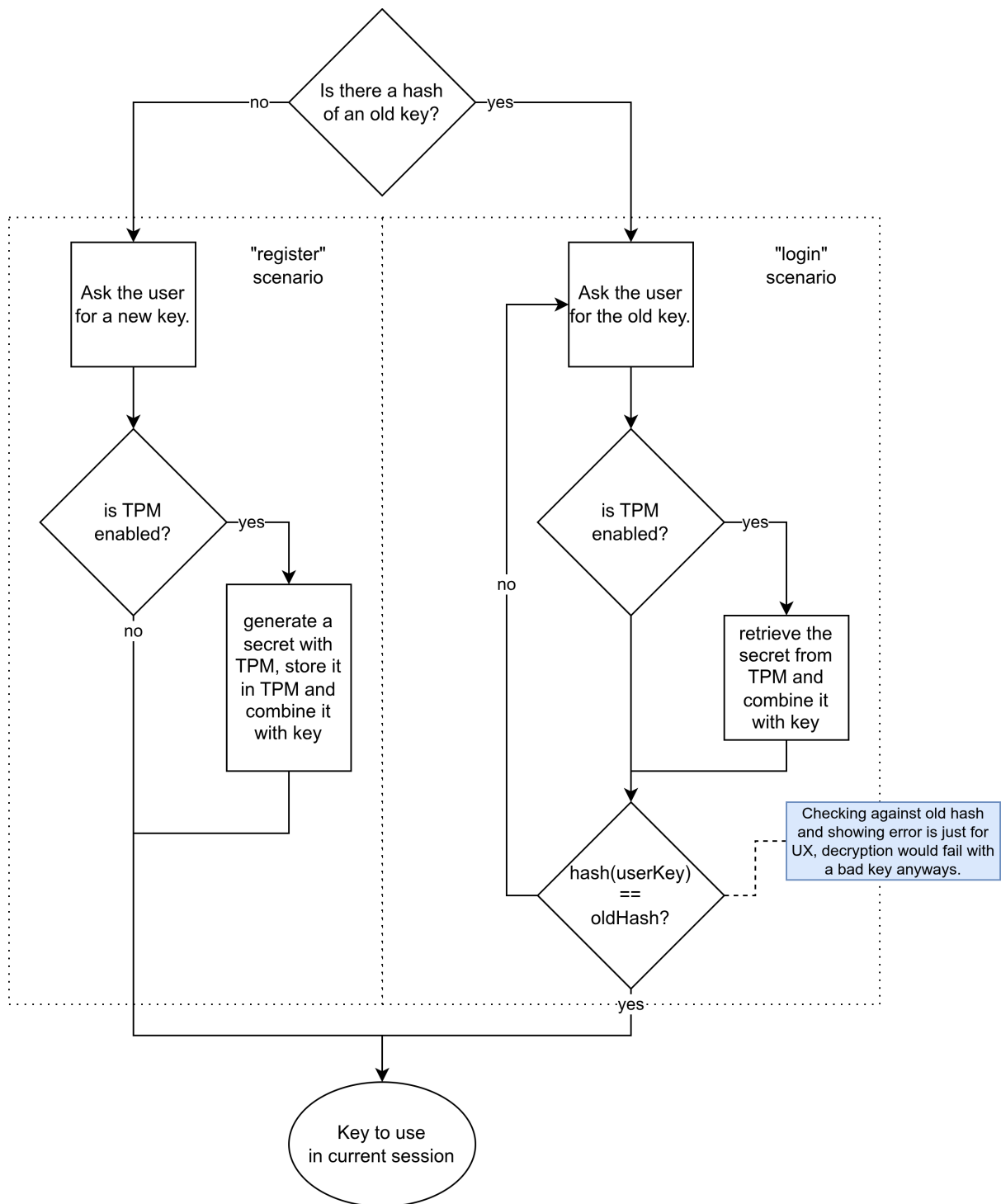


Figure 4.1: The two main flows of the proposed framework.

The main issue of USB sticks (in our context) is that they are not read-only media. Someone could steal the USB stick without our knowledge, and copy its contents. Stealing and putting back the USB stick could be done in less than a minute, making this kind of attack easy to execute. The attacker could then brute-force breaking the USB without any time limitations. Or even worse, if the attacker gets to know our password, he/she can break in without having our media, using the copy. Nevertheless, USB Encryption is a good first line of attack against not-so tech-savvy attackers.

4.1.2 User Password

Aside of the password for the USB's encryption, the framework also requires a user password. It is recommended for this password to be different from the password provided for the USB's encryption, but a check for difference is out of scope. So it is up to the user what password it chooses for the application. We will reference to this as the User Password from now on.

You might ask, what is the purpose of the User Password? It is used to encrypt the internal storage of the application, i.e., the secrets. While the application's source code, including the storage, is encrypted at rest due to USB Encryption, this encryption ensures the passwords are encrypted even when the app is running. This protects against "coffee-shop" attacks: the user might leave their device unlocked and unsupervised, and an attacker could quickly read the secrets from their device. There are other use-cases of the user password as well, described in later sections.

The user password is not stored. Instead, the hash of the user password is stored inside a file. If this file does not exist, it is considered that no user password was provided beforehand, so the user is asked to enter a new password. If this file does exist at start, the user shall provide the old password, which is verified against the existing hash. An incorrect password prevents the start of the flow, since the system has no knowledge about the password, and without it, it cannot decrypt the storage, to access the stored secrets.

4.1.3 TPM

The read-write nature of a USB cannot be changed, and making a USB stick uncopyable is either impossible or out of scope. The encryption with the help of the user password suffers the same vulnerability as the USB encryption, it is just another layer, but breakable in the same fashion. Detecting if another TOTP Application is using the secret (for example, if the attacker stole the secret and introduced it into a TOTP application) is also out of scope. But, something had to be done against the attack scenario presented in Section 4.1.1. This is where the TPM chip comes into discussion, and where we start to exchange convenience for security.

Our framework uses the TPM to get a secret which only one machine can provide, and mix it with the user-provided key. How the secret is mixed with the user password is up to the encryption algorithm. The most straight-forward option, and also the one implemented by Authenstickator, is to use the secret as a salt.

Initially, this secret does not exist. It has to be random, unpredictable, and long. So, the TPM is used to generate it. This provides hardware-backed true randomness. The secret is then stored inside the TPM's NVRAM. This is non-volatile memory, meaning

that it will be present after reboot. Upon each start, the NVRAM is queried for the secret.

Due to the nature of the TPM chip, the random number cannot be guessed. It is 16 bytes long, so brute-forcing it is not feasible. Combining with the user password makes it impossible for other applications using the TPM (for example, malware) to decrypt the storage. If the combination is properly made (ex: secret used as salt for the key), brute-forcing the decryption will also not be feasible.

The downside of using TPM is the same as the main upside: it ties the application to a specific machine. If the user uses one machine all the time, it is not a problem. If the user switches between multiple devices (which support USBs), he/she might consider not using TPM. The framework marks TPM usage optional: if disabled, simply the user password is used for storage encryption.

4.2 The Implementation

The implementation of the proposed framework presented in Section 4.1 is the application called Authenstickator. It consists of a Python front-end and a simple web front-end (HTML, CSS, JavaScript). The architecture of Authenstickator can be seen on Figure 4.2. The front-end and the back-end is decoupled, resulting in a modular architecture. In the following, detailed description of both will come.

4.2.1 The Back-end

As mentioned earlier, the back-end of the application is written in Python. While it might sound like a weird choice for a security-first application, there are reasons for this decision. First and foremost, Python provides a lot of libraries, making implementation easier. Using standard implementations and not reinventing the wheel is usually the better approach for small teams; and especially in the case of cryptographic algorithms, like encryption and hashing.

Second of all, Python provides an easy way to manage dependencies. With the help of the `pip` package manager, any library can be easily downloaded. All the libraries were downloaded inside a virtual environment (`venv`), so they are isolated and do not mess with the system's packages. The downloaded packages were then listed inside a `requirements.txt` file, so one can install all of them with a single command.

While Python isn't famous about its OOP capabilities, I chose an object-oriented approach. For me and for most developers, it is simply a thing of habit to think of complex problems in terms of classes, objects, and interactions between objects. It provides structure, and a common language among developers. In Authenstickator, every Python file contains exactly one class, kind of like how Java enforces it.

Python is also duck-typed, meaning that for Python, "if something quacks, it is a duck". It does not enforce types, there is no compilation. However, types can be linted, and I naturally chose to do so. More specifically, in each method's signature, I also specified what data type I expect for each parameter, and what the method should return. This is not enforced by Python, but a good IDE (like PyCharm) can catch a lot of bugs this way. Also, it is clearer to understand the intentions of methods.

The backend consists of several packages, which will be presented in the next sections. Each package is independent of the other. They can be (and are) unit tested; see: Section 5. They will be presented in the following.

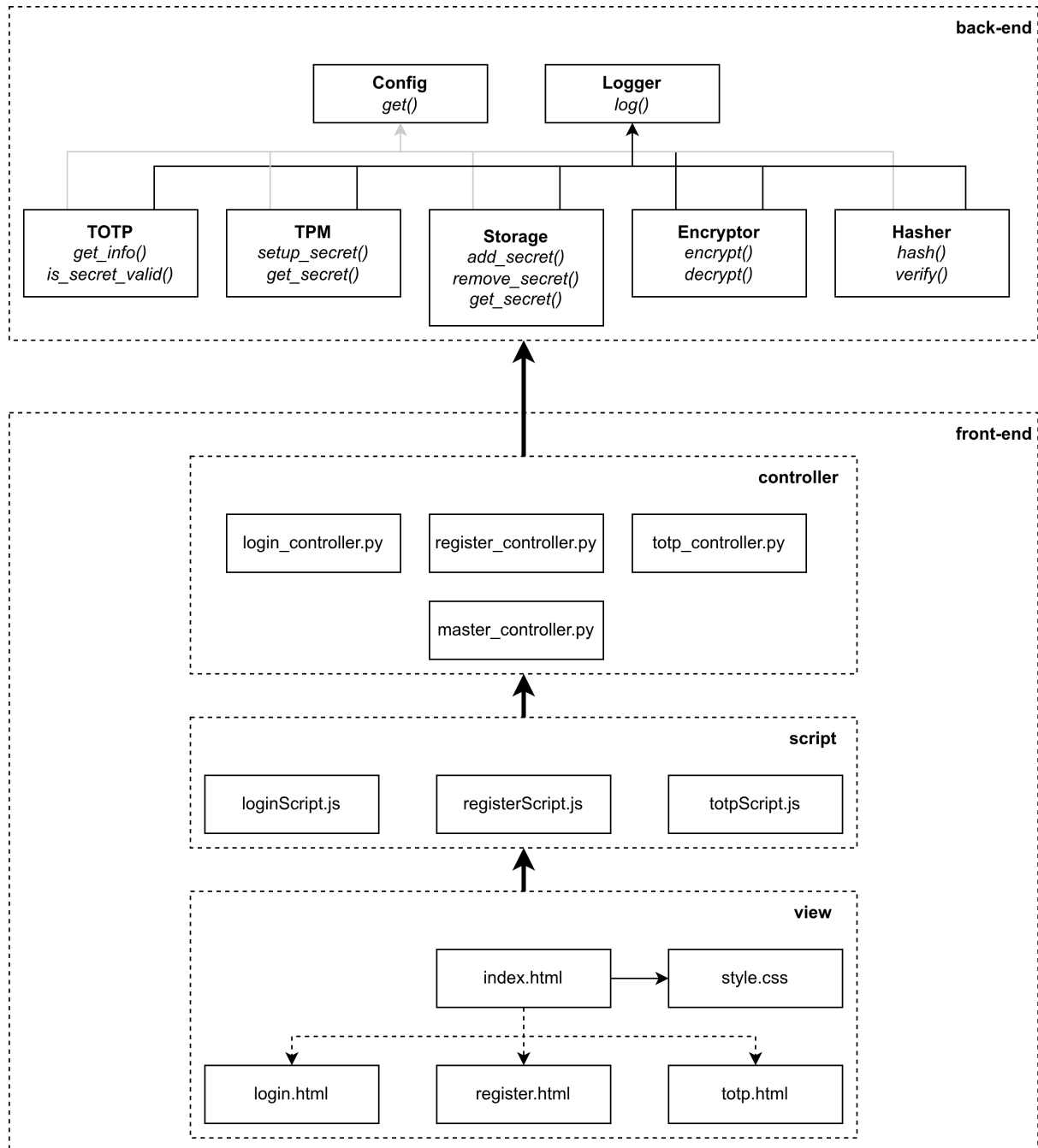


Figure 4.2: High-level diagram of the architecture.

Config

The Config package contains the `ConfigManager`, as well as one or two JSON files. The `ConfigManager` is a singleton class, and its instance provides the `get` method. The role of this class and its method is to read the JSON configuration file and provide configuration values for other back-end classes. This way, the developer (or a more tech-savvy user) can easily change the parameters (configs) of Authenstickator.

The name of the config file is hardcoded, and it is `config.json`. It provides a multi-layer config file, where configs are grouped by the package/ feature they are used by. The developers also have the possibility to define a `config-local.json` file, which if present, has priority over `config.json`. This is also part of `.gitignore`, i.e., not in version control. So it is solely for local development, avoiding pushing to git test configurations.

An example `config.json` looks like this:

```
1 {
2   "logger": {
3     "log_level": "DEBUG",
4     "log_file_path": "log.txt"
5   },
6   "pywebview": {
7     "debug": false
8   },
9   "tpm": {
10    "enabled": true,
11    "fapi_profile_name": "P_RSA2048SHA256",
12    "fapi_profile_file": "P_RSA2048SHA256.json",
13    "fapi_profile_url": "https://raw.githubusercontent.com/
14      tpm2-software/tpm2-tss/3.2.0/dist/fapi-profiles/
15      P_RSA2048SHA256.json",
16    "virtualized": false,
17    "virtualized_tpm_path": "/tmp/vtpm",
18    "virtualized_tpm_port": 2321,
19    "virtualized_tpm_ctrl_port": 2322,
20    "virtualized_tpm_startup_seconds": 1
21  },
22  "storage": {
23    "storage_file_path": "storage.txt",
24    "hashed_user_password_file_path": "hashed-user-password.
25      txt"
26  },
27  "encryptor": {
28    "type": "AES"
29  },
30  "hasher": {
31    "type": "ARGON2"
32  }
33 }
```

The meaning of each configuration parameter will be explained in the corresponding package's section.

Config Name	Allowed Values	Meaning
<code>logger.log_level</code>	"ERROR", "WARNING", "INFO", "DEBUG"	The lowest log level which shall be logged.
<code>logger.log_file_path</code>	A valid path, like "log.txt"	The path at which the log file shall be saved.
<code>pywebview.debug</code>	true, false	If pywebview should run in debug mode or not.
<code>tpm.enabled</code>	true, false	If the TPM (and related functionalities) shall be used.
<code>tpm.fapi_profile_name</code>	todo	todo
<code>tpm.fapi_profile_file</code>	todo	todo
<code>tpm.fapi_profile_url</code>	todo	todo
<code>tpm.virtualized</code>	todo	todo
<code>tpm.virtualized_tpm_path</code>	todo	todo
<code>tpm.virtualized_tpm_port</code>	todo	todo
<code>tpm.virtualized_tpm_ctrl_port</code>	todo	todo
<code>tpm.virtualized_tpm_startup_seconds</code>	todo	todo
<code>storage.storage_file_path</code>	todo	todo
<code>storage.hashd_user_password_file_path</code>	todo	todo
<code>encryptor.type</code>	todo	todo
<code>hasher.type</code>	todo	todo

Logger

The logger package contains `LoggerManager`, yet another singleton class. As its name suggests, the logger manager provides a logger instance for logging. The standard logging library is used. For simplicity, Authenstickator uses only four log levels. The log level can be set by changing the value of the config `logger.log_level`.

- **Error:** for fatal issues, ones which break execution.
- **Warning:** for issues which should not be ignored, but do not crash the app.
- **Info:** for messages which are not issues, but things that might help in audit.
- **Debug:** the most low-level messages; these are useful only for development.

While the logger is centralized, logging happens in two places: the console and the log file. The same log lines are printed in both outputs, but the log file persists, meaning that new logs are appended to the old ones instead of starting a new file at each run. This is good for debugging and audit.

The format of log lines is: date and time, severity level in a letter (E, W, I, D), the file and the line of the source, the logger, and the message itself. For example:

```
[2026-08-16 22:14:47] [E] [util.py:208] [pywebview] Error while
processing window.native.bin: unable to get the value
```

While there is only one logger instance, `pywebview`, the front-end library also logs its warnings and errors. These are useful, so instead of throwing them out, they are “propagated” to our logger. This is why the logger is also printed in the log line: it can be either `[root]` or `[pywebview]`.

As a convention, each class instantiates a logger instance outside its constructor, for simplicity. Methods which handle requests from the UI often log on debug level entrance in them. As a general rule, secrets and sensitive data shall not be logged. Logging such information would make encryption of the storage broken.

TOTP

TODO

TPM

TODO

As explained in [4], on Linux, TPM shows up as device nodes. `/dev/tm0` is the Hardware Interface, while `/dev/tpmtm0` is the Kernel Resource Manager. Most applications use the Kernel Resource Manager, since through that entry the kernel handles context management and other low-level mechanisms.

TSS stands for Trusted Software Stack, and it is the software around the TPM chip. It has a layered architecture, providing programmer-friendly API calls. Its layers are specifications, and there are different implementations of it. The four layers are:

1. **FAPI**: Feature API. High-level, developer-friendly. No need to handle sessions and authorization.
2. **ESAPI**: Enhanced System API. Wraps sessions and handles retries when TPM is busy.
3. **SAPI**: System API. Low-level C interface implementing the tpm2.0 specifications.
4. **TCTI**: TPM Command Transmission Interface. The Transport Layer, working with bytes.

Authenstickator uses the `tpm2_pytss` library and the FAPI layer. FAPI was chosen since it is the most high-level, so the resulting code will be more generic. Going lower in the TSS layers would mean more OS/ hardware specific code, and more chances to make errors.

When this layer is used, configurations are held in a file named `fapi-config.json`. The default location is `/etc/tpm2-tss/fapi-config.json`, but it can be overwritten by setting the `TSS2_FAPICONF` environment variable. Authenstickator generates one such file specifically for the usage of this app, so that it does not mess with system-wide TPM settings. It might look something like this:

```

1 {
2     "profile_name": "P_RSA2048SHA256",
3     "profile_dir": "/home/broland/.local/share/
4     authenstickator/profile-dir",
5     "user_dir": "/home/broland/.local/share/authenstickator/
6     user-dir",
7     "system_dir": "/home/broland/.local/share/authenstickator
8     /system-dir",
9     "system_pcrs": [],
10    "log_dir": "/home/broland/.local/share/authenstickator",
11    "ek_cert_less": "yes",
12    "tcti": "swtpm:port=2321"
13 }

```

The FAPI profile is downloaded from the internet, from the URL specified in the config file. The profile's version shall match the version of the installed native `tpm2-tss/libtss2-fapi`. This can be retrieved with the help of the command `apt policy libtss2-fapi1`. Note that this is different from the `tpm2-pytss` version shown in `requirements.txt`.

Storage

TODO

Encryptor

The AES algorithm is a symmetric one, meaning it uses the same key for encryption and decryption. It does not accept any key, it has to be 16, 24 or 32 bytes long. Requiring the user to choose a key with a fixed length is tedious, so Password-Based Key Derivation Function 2 (PBKDF2) is used. This mixes its input with a salt, and outputs a fixed length string, making it perfect for our use case.

TODO

Hasher

TODO

4.3 Provisioning the TPM: a Requirement for the Application

While the application is bundled with its dependencies, the usage of TPM comes with a cost. These chips need to be provisioned, which is a one-time operation to make them run correctly. Provisioning a TPM should be an easy process, but it is rather difficult in practice. Since provisioning might require authentication, it cannot be done by an automatic script. Also, if you mess too much with your real TPM, you might get locked out, just like I did.

If you run a Debian-based distribution, you might try installing with `apt` the `tpm2-tools` package. After installation, try running the `sudo -u tss tss2_provision`

command. It shall create a default FAPI configuration file at path `/etc/tpm2-tss/fapi-config.json`, and automatically download some profiles in `/etc/tpm2-tss/fapi-profiles/`. For more details about this JSON, see Section 4.2.1.

Unfortunately, the `tpm2-tools` package from `apt` did not work for me, and might not work for you neither. If this is the case, you can try running the `setup_fapi.sh` script.

4.4 Development

Development was done on a Linux machine, namely on Pop!_OS 22.04 LTS. The development environment described below was tested on a virtual machine with a fresh Ubuntu 22.04.5 LTS installation. As a result, the following commands will probably work on Debian-based Linux distributions. The IDE used throughout development was PyCharm IDE, but other editors shall work as well.

4.4.1 Basic Setup

First, ensure that your system has Python. Use the following command:

```
python3 --version
```

Both the development machine and the test VM used Python 3.10.12, which came with the OS installation. So, if you have Python-related issues, try this specific version.

Then, Git is used for version control. The source code of Authenstickator is available on GitHub as a public repository, and we will clone the project from there.

```
sudo apt install -y git
git clone https://github.com/broland29/authenstickator.git
cd authenstickator
```

Next, you will need to create a virtual environment and activate it. A virtual environment (`venv`) is used to isolate Python packages used for Authenstickator from other Python packages on the system. This is standard dependency management, and ensures not just that Authenstickator works well, but that we don't break other apps. Each developer creates its own virtual environment and these are not pushed to git (specified in `.gitignore`).

```
sudo apt update
sudo apt install -y python3.10-venv
python3 -m venv venv
source venv/bin/activate
```

Before installing the Python dependencies from `requirements.txt`, there are other Linux packages required for the Python packages to work. These include: a build compiler, the TPM2 development headers and build tools, some FAPI requirements (TPM-related), and some PyGObject requirements (`pywebview`/ UI related). For development, to avoid TPM lockout, a software TPM, `swtpm` is recommended, which also needs installation. Be careful with the following command: if you accidentally leave spaces after the line-breaking backslashes, it won't work.

```
sudo apt install -y git \
    libtss2-dev pkg-config python3-dev \
```

```
libssl-dev libjson-c-dev libcurl4-openssl-dev \
libgirepository1.0-dev libcairo2-dev libglib2.0-dev gobject-
introspection \
swtpm
```

Now that we have the dependencies ready, we can install the actual Python libraries Authenstickator needs. Since we have the virtual environment activated (signaled by the `(venv)` prompt), the libraries will be installed only inside this environment. The requirements are listed in `requirements.txt` and can be installed with the following command:

```
pip install -r requirements.txt
```

Once it is done installing, the app can be started. Remember that, for subsequent launches, you will also need to activate the virtual environment before starting the app.

```
python3 -m src.main
```

4.4.2 Testing with TPM on a VM

If you wish to test the application with the real TPM setting, but still want to avoid using your own TPM and getting locked out, you can use a Virtual Machine. My recommendation is QEMU/KVM, since it is easy to set up and works well. The following steps for TPM setup are for QEMU/KVM 4.0.0. Turn off the virtual machine if it is currently running. Inside the “Show virtual hardware details” panel (the icon containing the letter I, when the VM is selected), click “Add Hardware”. Select TPM. Inside “Advanced options”, select Model “CRB” and Version “2.0”. Click finish, and start the VM.

4.4.3 Setup TPM

First, check if you actually have a TPM chip with the following commands:

```
sudo dmesg | grep -i tpm
ls -l /dev/tpm0 /dev/tpmrm0
```

If the output contains log files like below, good news, you have a TPM chip and can move to the next step.



```
(venv) broland@broland:~/authenstickator$ sudo dmesg | grep -i tpm
[sudo] password for broland:
[ 0.012065] ACPI: TPM2 0x0000000007FFD2C3C 00004C (v04 BOCHS BXPC 00000001 BXPC 00000001)
[ 0.012076] ACPI: Reserving TPM2 table memory at [mem 0x7ffd2c3c-0x7ffd2c87]
[ 2.043231] tpm_crb MSFT0101:00: Disabling hwrng
(venv) broland@broland:~/authenstickator$ ls -l /dev/tpm0 /dev/tpmrm0
crw-rw---- 1 tss root 10, 224 aug 20 18:30 /dev/tpm0
crw-rw---- 1 tss tss 253, 65536 aug 20 18:30 /dev/tpmrm0
```

Figure 4.3: The `dmesg` logs contain such lines if TPM is present.

Unfortunately, the `tpm2-tools` package from `apt` does not work. You can try your luck with these commands though.

```
sudo apt install -y tpm2-tools
sudo -u tss tss2_provision
```

For me, this did not work, I got a command not found error. The alternative is to compile tpm2-tools from source. It is very fast and shall work out of the box.

```
sudo apt install -y build-essential autoconf autoconf-archive
    automake libtool pkg-config gcc git libssl-dev libcurl4-
    openssl-dev libjson-c-dev uuid-dev libtss2-dev
cd ~
git clone https://github.com/tpm2-software/tpm2-tools.git
cd tpm2-tools
./bootstrap
./configure --enable-fapi
make -j$(nproc)
sudo make install
sudo ldconfig
sudo -u tss tss2_provision
```

The current user has to be added to the tss group.

```
sudo usermod -aG tss "$USER"
```

Development was done with a simulated TPM. Initially I worked with the real TPM, and I got locked out when I was messing with it. A virtualized TPM makes experimenting easier and leaves the real TPM untouched.

The TPM emulator **swtpm** can be downloaded with APT:

```
sudo apt update && sudo apt install swtpm
```

It can be started by creating a temporary directory (will be deleted at system shutdown) and using it to run the virtual TPM. After this, the third command sets the variable **TSS2_TCTI**, so that in the current terminal session the virtualized TPM replaces the real one.

```
mkdir -p /tmp/vtpm

swtpm socket --tpm2 --tpmstate dir=/tmp/vtpm --ctrl type=tcp,port
    =2322 --server type=tcp,port=2321 --flags not-need-init &

export TSS2_TCTI="swtpm:port=2321"
```

To check manually if swtpm is running, you can netcat it.

```
nc -zv localhost 2322
```

If anything goes wrong, the virtual TPM can be stopped easily by killing its process, and the folder can be deleted.

```
pkill swtpm | rm -r /tmp/vtpm
```

FAPI needs a one-time initialization, called provisioning. This can be done with the **provision** method. To avoid error **0x0006001f (TSS2_FAPI_RC_PATH_ALREADY_EXISTS)**, this method is called with parameter **is_provisioned_ok** set to true. This way the application does not fail on subsequent startups, or if the user provisioned the TPM beforehand manually or through another application.

The standard development starts with cloning the project.

To use `tpm2_pyttss` and its FAPI component, you need to install a build compiler; the TPM2 development headers and build tools; some FAPI requirements; and some PyGObject requirements, which are dependencies for pywebview. For the software TPM, `swtpm` is needed.

```
sudo apt install git \
    python3.10-venv \
    build-essential \
    libtss2-dev pkg-config python3-dev \
    libssl-dev libjson-c-dev libcurl4-openssl-dev \
    libgirepository1.0-dev libcairo2-dev libglib2.0-dev gobject-
    introspection \
    swtpm
```

Careful with trailing whitespaces after that might break your command.

```
git clone https://github.com/broland29/authenstickator.git

cd authenstickator

python3 -m venv venv

source venv/bin/activate

pip install -r requirements.txt

python3 -m src.main
```

To ensure that the app is properly working, you can run the tests:

```
python -m pytest
```

The app is run as a module (hence the `-m` flag) and started from the root folder. This means that imports have to start from the `src` folder, for example, from `src.config.config_manager` instead of `from config.config_manager` import `ConfigManager`. While the latter might work in PyCharm due to it modifying the `PYTHONPATH` in the back.

If you end up adding new dependencies, run:

```
pip freeze > requirements.txt
```

Common errors: `tpm2_pyttss.TSS2_Exception.TSS2_Exception: tcti:IO failure: you are probably running with virtualized TPM enabled, but the virtualized TPM is not running.`

The command to create a USB export:

We have to specify paths to point to `.`, so that imports work correctly. We put the `dist/build/spec` paths to the temporary folder, to not clutter the project folder (messes with git and such). Spec path is not set, so the `.spec` file is generated in the default location, where it is executed (in the project root). This is intentional, since `add-data` will take paths relative from the `.spec` file. JSON and HTML files are not added by default and need to be added with `add-data`. See `pyinstaller -help` for more details.

```
pyinstaller src/main.py \
    --clean \
```

```
--add-data "src/config/config.json:src/config" \  
--add-data "src/ui/view:ui/view" \  
--add-data "src/ui/script:ui/script" \  
--paths . \  
--distpath /tmp/authenstickator/dist \  
--workpath /tmp/authenstickator/build
```

There is always a key, no matter if encryption or tpm is disabled.

Max line length is 100 characters.

Making real TPM work: 1. Make sure you have a TPM2 chip:

```
sudo dmesg | grep -i tpm
```

2. Add your user to the tss group.

```
sudo usermod -aG tss "$USER"
```

3. Restart.

4. Run the app (provisions in the back)

Testiing with VM: - start from fresh install - add to group - add TPM - add stick

Chapter 5

Theoretical and Experimental Results

5.1 Testing on Linux

Since the most popular distro is Ubuntu¹, I chose to test and build on a Ubuntu 22.04 LTS machine. Ubuntu 20.04 LTS hit its Standard Security Maintenance deadline at May 2025, while 22.04 LTS is still in free support until May 2027. Distros shall be backward compatible, so the results shall be the same for newer Ubuntu releases as well, and the setup will probably work on Debian and other Debian-based distros as well.

5.2 Unit tests

For unit tests, the `pytest` library was used. To run all tests, with the virtual environment activated, run the following command:

```
python -m pytest
```

¹2025 Stack Overflow Developer Survey: <https://survey.stackoverflow.co/2025/technology>

Chapter 6

Conclusions

Bibliography

- [1] Fortinet, “What is two-factor authentication (2fa), and how can it be enabled?” <https://www.fortinet.com/resources/cyberglossary/two-factor-authentication>, accessed: August 8, 2026.
- [2] D. M’Raihi, S. Machani, M. Pei, and J. Rydell, “TOTP: Time-Based One-Time Password Algorithm,” RFC 6238, May 2011. [Online]. Available: <https://www.rfc-editor.org/info/rfc6238>
- [3] W. Arthur, D. Challener, and K. Goldman, *A Practical Guide to TPM 2.0*. Apress, 2015.
- [4] pi3g, “From application to tpm - linux deepdive,” <https://www.youtube.com/watch?v=PF5Uv16kJ-M>, October 2025, accessed: August 8, 2026.